Esercizio S2/L4

Introduzione.....2
Descrizione dell'Ambiente di Lavoro..... 2
Struttura e Funzionamento dello Script Python.....3
Esecuzione e Risultati..... 5

Introduzione

L'obiettivo dell'esercizio è realizzare, configurare ed eseguire un assistente per la conversazione basato sulle API Gemini di Google, operando all'interno di un ambiente virtualizzato Linux (Kali Linux su VirtualBox). L'attività si inserisce nel contesto dello studio delle tecnologie di intelligenza artificiale generativa e della loro integrazione in applicazioni Python.

L'esercizio prevede due fasi principali:

1. **Sviluppo dello script Python** per la gestione del dialogo.
2. **Esecuzione e verifica funzionale** del sistema tramite interazione diretta da terminale.

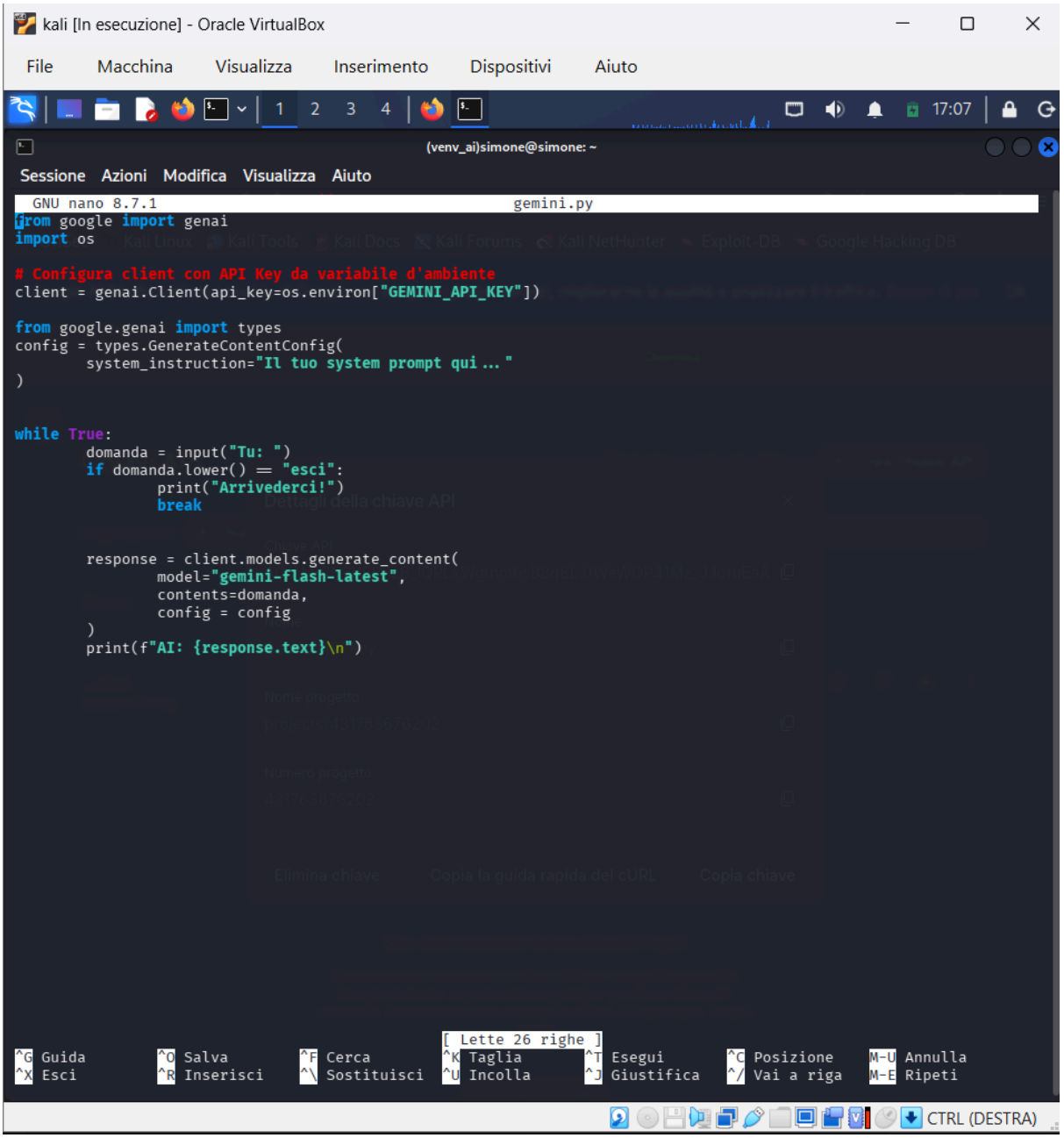
Questa attività permette di comprendere sia gli aspetti tecnici legati all'uso delle API, sia le dinamiche di progettazione di un'interfaccia.

Descrizione dell'Ambiente di Lavoro

L'esercizio è stato svolto in un ambiente virtualizzato composto da:

- **Oracle VirtualBox** come hypervisor.
- **Kali Linux** come sistema operativo guest.
- **Python 3** con ambiente virtuale dedicato (`venv_ai`).
- Libreria **google-genai** per l'interazione con il modello Gemini.

L'utilizzo di una macchina virtuale consente di isolare l'ambiente di sviluppo, garantendo riproducibilità e sicurezza durante l'esecuzione di codice sperimentale.



Struttura e Funzionamento dello Script Python

Lo script `gemini.py` implementa un semplice ciclo conversazionale basato sui seguenti elementi:

Configurazione del Client:
Il programma importa la libreria `google.genai` e inizializza il client utilizzando una **API Key** caricata da variabile d'ambiente (`GEMINI_API_KEY`). Questa scelta garantisce una gestione sicura delle credenziali, evitando l'inserimento diretto nel codice.

Definizione del System Prompt:
Viene configurato un oggetto `GenerateContentConfig` contenente un'istruzione di sistema.

Il system prompt stabilisce il comportamento generale del modello, definendo tono, ruolo e vincoli della generazione.

Ciclo Conversazionale:

Lo script implementa un loop `while True` che:

1. Richiede all'utente un input testuale.
2. Interrompe l'esecuzione se l'utente digita *"esci"*.
3. Invia la richiesta al modello tramite `client.models.generate_content`.
4. Stampa la risposta generata.

Questa struttura riproduce un'interazione in stile chatbot, mantenendo semplicità e chiarezza.

```
┌──(venv_ai)─(simone㉿simone)-[~]
└─$ python gemini.py
Tu: ciao
AI: Ciao! Come posso aiutarti oggi?

Tu: spiegami in poche parole la triade CIA della cyber security per inserirla all'interno di una slide powerPoint
AI: Ecco una struttura sintetica ed efficace, perfetta per una slide:

---

### **La Triade CIA: I 3 Pilastri della Cyber Security**

*   **Riservatezza (Confidentiality):** Garantire che i dati siano accessibili **solo a chi è autorizzato**. (Es. C
rittografia, Password).
*   **Integrità (Integrity):** Assicurare che le informazioni **non vengano alterate** o cancellate da soggetti non
 autorizzati. (Es. Firme digitali).
*   **Disponibilità (Availability):** Garantire che sistemi e dati siano **sempre pronti all'uso** quando necessari
o. (Es. Backup, Protezione anti-DDoS).

---

**Suggerimento per il layout:**
Puoi disporre questi tre punti ai vertici di un **triangolo** con al centro la parola "SICUREZZA". Questo rende il
concetto visivamente immediato.

Tu: Grazie mille !
AI: Prego! È stato un piacere aiutarti.

Se hai bisogno di altro, sono qui. Buona giornata!

Tu: esci
Arrivederci!

┌──(venv_ai)─(simone㉿simone)-[~]
└─$
```

Esecuzione e Risultati

Durante l'esecuzione dello script, l'utente ha avviato una conversazione con il modello.

L'interazione mostrata riguarda la richiesta di una **spiegazione sintetica della triade CIA** (Confidentiality, Integrity, Availability) per l'inserimento in una slide PowerPoint.

Il modello ha fornito:

- una definizione chiara e strutturata dei tre principi fondamentali della cybersecurity;
- un suggerimento grafico per la presentazione (disposizione a triangolo).

Il comportamento osservato conferma la corretta configurazione del client, la funzionalità del ciclo conversazionale e la capacità del modello di generare contenuti coerenti e contestualizzati.

Conclusione

L'esercizio ha permesso di sviluppare e testare un assistente per la conversazione basato sulle API Gemini, verificandone il funzionamento in un ambiente virtualizzato.
L'interazione osservata conferma l'efficacia del modello nel generare risposte strutturate e contestualizzate, evidenziando il potenziale dell'intelligenza artificiale generativa come supporto operativo e formativo.